

Low Level Design — Complete Study Plan

Java · OOP · Design Patterns · Interview Preparation | Reference & progress tracker

How to use this. Every topic has a checkbox and a line explaining *why an interviewer cares* — not just what it is. Tick items as you finish them so you always know where you stopped. Topics marked **CRITICAL** are ones where a gap is likely to actually cost you an offer; **HIGH** ones come up in most rounds.

Estimated scope: 30–35 sessions at a steady pace. Phases 0–4 are foundations (roughly 12–15 sessions); Phase 5 is the bulk and runs ongoing; Phase 6 interleaves every ~5 sessions.

Method throughout: concept → you write the code → review against interview standards → an extensibility twist on the same problem. Retrieval practice (cold re-design with no notes) is scheduled deliberately, because re-reading feels like learning and isn't.

Phase 0 — Java Fluency for Design Work

2–3 sessions. Goal: stop fighting the language so all your attention goes to design. You already have programming logic from C++/DSA — this is remapping, not learning to program.		
✓	Topic	Why it matters in an interview
☐	Syntax, types, arrays, control flow	Baseline. Fumbling <code>int</code> vs <code>Integer</code> or <code>.length</code> vs <code>.size()</code> mid-round wastes time and signals unfamiliarity with the language you claimed.
☐	String vs StringBuilder, immutability	"Why is <code>String</code> immutable?" is a stock question. The real answer (string pool sharing, safe <code>HashMap</code> keys, thread safety) also introduces immutability as a design tool you'll use later.
☐	Methods, overloading, this	Overloading vs overriding is asked constantly and confused constantly. Compile-time vs runtime resolution is the distinction.
☐	Classes, constructors, static vs instance	Every LLD answer is classes. Static state is also the entry point to Singleton and to thread-safety discussions.
☐	Collections: List, Map, Set, Queue, Deque	Every design stores entities somewhere. Knowing <i>which</i> to pick (and the cost of each operation) is itself a design decision interviewers probe.
☐	<code>equals()</code> and <code>hashCode()</code> CRITICAL	The moment a custom object goes into a <code>HashSet</code> or becomes a <code>HashMap</code> key — which happens in nearly every LLD problem — default reference equality silently breaks lookups. The contract between the two, and why overriding one without the other is a bug, is a standard question.
☐	Enums (with fields and methods) HIGH	<code>VehicleType</code> , <code>SpotSize</code> , <code>OrderStatus</code> , <code>SeatCategory</code> , <code>Direction</code> — LLD is full of them. Using a raw <code>String</code> where an enum belongs reads as junior immediately. Java enums also hold state and behaviour, and are the idiomatic thread-safe Singleton.
☐	Generics basics <code><T></code>	Needed to read and write typed collections and to build reusable components (a generic <code>Repository</code> , a generic <code>Cache</code>).
☐	Exceptions, custom exceptions, try/catch	Real designs signal failure with exceptions, not print statements. Checked vs unchecked, and when to define <code>SeatAlreadyBookedException</code> , is a design conversation.
☐	Optional and null handling	"What if the spot isn't found?" Returning <code>Optional</code> instead of <code>null</code> is a small thing that reads as mature.
☐	Lambdas, functional interfaces, Comparator HIGH	Sorting objects by multiple criteria comes up constantly — bookings by time, spots by distance. <code>Comparator.comparing().thenComparing()</code> and method references are expected fluency in modern Java.
☐	Streams (light: filter, map, collect)	Not required, but writing a loop where a one-line stream would do is a small style ding, and you must be able to <i>read</i> streams in any real codebase.
☐	Static nested classes, varargs	Static nested classes are structurally required for the Builder pattern. Varargs shows up in convenience APIs.
☐	Immutability: final fields, defensive copying	Immutable objects are automatically thread-safe — which is the cheapest answer to a concurrency follow-up. Also: how to make a class properly immutable is a direct question.

Phase 1 — Core OOP for Design

2–3 sessions. Goal: the judgement that decides whether your class diagram is good or bad. This is where most of the actual signal in an LLD round comes from.		
✓	Topic	Why it matters in an interview
☐	Encapsulation done properly	Public fields make validation decorative — anyone can bypass it. Interviewers scan your field declarations early; private-by-default is the expected instinct.
☐	Inheritance: extends, super, overriding	Mechanics you need before you can judge when <i>not</i> to use it. Also: private parent fields are invisible to children — a common stumble.
☐	Composition over inheritance CRITICAL	The single highest-signal judgement in LLD. Inheritance is compile-time and permanent; composition is runtime and swappable. Interviewers deliberately set traps where inheritance causes a class explosion, and watch whether you notice.
☐	Interfaces vs abstract classes CRITICAL	A guaranteed question. The precise answer: interface = capability that unrelated types may share, abstract class = family sharing real state. Default to interface; abstract class only when shared state forces it. Post-Java-8 follow-up: "why do abstract classes still exist?" → state.
☐	Polymorphism (compile-time vs runtime)	The mechanism every design pattern is built on. If you can't explain runtime dispatch, you can't explain why Strategy works.

☐	Access modifiers, final	The set of public methods <i>is</i> your class's contract. Keeping it small is design; exposing everything is not.
☐	Coupling and cohesion	The vocabulary interviewers use when giving feedback. You should be able to name why a design is bad, not just feel it.
☐	Association / aggregation / composition	Needed <i>while</i> designing, not later — you'll be asked "does the Order own the Payment, or just reference it?" and the answer changes lifecycle and deletion semantics.
☐	Law of Demeter	<code>a.getB().getC().doThing()</code> is a recognised smell. Knowing the name lets you justify a refactor out loud.

Phase 2 — SOLID and the Counterweights

2 sessions. Format: a deliberately bad class → you identify the violation → you refactor. Naming principles out loud during a round is a large part of the score.

✓	Topic	Why it matters in an interview
☐	Single Responsibility	The most common real violation: a God class that parses, validates, calculates and prints. Easy to spot, easy to fix, and interviewers plant it on purpose.
☐	Open/Closed HIGH	"How would you add a new payment type?" is the standard extensibility probe. Good answer: write one new class, change nothing else. Bad answer: add an else if.
☐	Liskov Substitution	Explains precisely why a FixedDepositAccount that refuses withdrawals shouldn't extend BankAccount. The classic Square/Rectangle example is worth having ready.
☐	Interface Segregation	Justifies splitting a fat interface — e.g. why Bonusable is separate from Employee rather than every employee being forced to implement a bonus they don't get.
☐	Dependency Inversion HIGH	"Depend on abstractions" is what makes your design testable and swappable. This is also literally what Spring's DI container does — connect it to what you already know.
☐	DRY, KISS, YAGNI	The counterweight. Candidates who just learned patterns bolt three onto a problem that needed one, and it is marked down as over-engineering. Knowing when a plain class is correct is part of the skill.
☐	Recognising over-engineering	Being able to say "a Factory here would be premature — one class implements this today" scores higher than adding the Factory.

Phase 3 — Design Patterns

4-5 sessions. Each pattern: the problem it solves → small coded example → where it shows up in real systems. Weighted by interview frequency, not treated as 23 equals.

Frequency reality check. Strategy, Factory, Observer, Singleton, Builder, State and Decorator cover roughly 85% of what you'll be asked to produce or recognise. Visitor and Flyweight are mostly trivia — know the one-line definition, don't over-invest.

Creational

✓	Pattern	Interview relevance
☐	Singleton HIGH	Asked mostly to probe <i>thread safety</i> : double-checked locking, volatile, why the enum singleton is the clean answer. Also expect "why is Singleton considered an anti-pattern?" — testability and hidden global state.
☐	Factory Method HIGH	The standard answer to "where do these objects get created?" Keeps new ConcreteType() out of business logic.
☐	Abstract Factory	Families of related objects. Less common than plain Factory but asked by name.
☐	Builder HIGH	The fix for a constructor with eight parameters. Very common in machine coding rounds where entities have many optional fields.
☐	Prototype	Lower priority. Know cloning and deep vs shallow copy.

Structural

✓	Pattern	Interview relevance
☐	Adapter	Integrating a third-party API whose interface doesn't match yours — a realistic scenario interviewers like.
☐	Decorator HIGH	Adding behaviour without subclass explosion — pizza toppings, coffee add-ons, discount stacking. Directly answers the combinatorial problem inheritance creates.
☐	Facade	Simplifying a complex subsystem behind one entry point. Often the right shape for a "Service" class.
☐	Composite	Tree structures treated uniformly — file systems, org charts, nested menus, comment threads.
☐	Proxy	Lazy loading, access control, caching. Also what Spring AOP actually does under the hood — worth connecting.

Behavioural

✓	Pattern	Interview relevance
☐	Strategy CRITICAL	The most frequently applicable pattern in LLD. Pricing rules, payment methods, matching algorithms, eviction policies, parking fee models — all Strategy.
☐	Observer CRITICAL	Any "notify users when X happens" requirement. Appears in almost every booking, ordering or alerting problem.
☐	State HIGH	Vending machines, elevators, order lifecycles, traffic signals. The clean alternative to a sprawling switch on a status field.
☐	Command	Undo/redo, request queuing, task scheduling. Asked directly in editor and remote-control style problems.
☐	Chain of Responsibility	Approval workflows, middleware, logging levels, ATM note dispensing.

☐	Template Method	Parent defines the skeleton, children fill one step. Common and easy to demonstrate.
☐	Iterator, Mediator, Memento	Recognition level. Memento pairs with Command for undo.
☐	Null Object	Small, but it's the clean alternative to scattered null checks — good to mention when an interviewer sees one.
☐	Visitor, Flyweight	Definition only. Rarely required to produce.

Phase 4 — Engineering Concerns Around the Design

3-4 sessions.

These are the follow-up questions that separate a passing answer from a strong one. Most candidates prepare patterns and neglect this entire phase.

Highest-risk gap.

Concurrency is the most common follow-up in booking-style problems and the most commonly under-prepared. "Two users book the same seat at the same time — what happens?" needs a real answer, not a hand-wave.

✓	Topic	Why it matters in an interview
☐	Concurrency in LLD CRITICAL	synchronized, ConcurrentHashMap, AtomicInteger, ReentrantLock; optimistic vs pessimistic locking; where deadlock comes from and how lock ordering avoids it. Needed for BookMyShow, Parking Lot, Ride-sharing, Rate Limiter, Wallet.
☐	Exception and error design	Checked vs unchecked; when a custom exception is worth defining; when returning a result object beats throwing. Shows you think about the caller.
☐	Dependency injection and testability HIGH	"How would you unit test this?" is a decoupling probe in disguise. Constructor injection lets you pass a fake — if you can't, your design is too coupled. You already do this in Spring; make it explicit.
☐	In-memory persistence / Repository HIGH	Every LLD problem needs storage and no interviewer wants a database. The expected shape is a Map-backed repository behind an interface.
☐	Layering: entity / service / repository	Gives your design a recognisable structure instead of a pile of classes. Maps directly onto what Spring's stereotypes already encode.
☐	Requirements: functional vs non-functional	The first five minutes. Diving into code without clarifying scope is one of the most common failure modes, and it's visible immediately.
☐	Identifying entities, actors, relationships	The bridge from a vague prompt to a class diagram. Nouns tend to be classes, verbs tend to be methods — a crude but effective starting heuristic.
☐	UML class diagrams and multiplicity	You will be asked to draw one, quickly, on a whiteboard or in Excalidraw. Correct arrow notation is a small skill that's easy to neglect until the round itself.
☐	Thinking out loud while designing HIGH	Trips up more candidates than the code does. Silence reads as being stuck; narrating tradeoffs reads as competence even when you're mid-decision.

Phase 5 — LLD Problems

Ongoing — the bulk of the work.

Each problem: clarify requirements → class design → Java implementation → review → an extensibility twist to test whether the design actually holds.

Two formats, prepared separately

Format	What it is and how to prepare
Machine coding <i>Flipkart, Swiggy, Uber, Zomato, Razorpay, Navi and similar</i>	90-120 minutes to build a <i>working, runnable</i> console application, often with tests. Graded on working code, clean structure and extensibility. IDE fluency and typing speed genuinely matter here — practise in IntelliJ with a timer, not in a chat window.
LLD discussion <i>Most large product and service companies</i>	45-60 minutes of class diagrams, tradeoff discussion and partial code. Graded on communication and design reasoning as much as on code. Practise narrating and drawing, not just implementing.

Problem list — roughly increasing difficulty

✓	Problem	What it exercises
☐	Tic-Tac-Toe	Warm-up class modelling. Board, Player, Game loop, win detection.
☐	Snake and Ladder	Entity modelling, dice abstraction, game state.
☐	Deck of Cards / card game	Enums doing real work, shuffling, Comparator for ranking.
☐	Parking Lot CRITICAL	The canonical problem. Strategy for fees, Factory for spot allocation, concurrency on spot assignment. Expect it anywhere.
☐	Vending Machine HIGH	State pattern, textbook case. Inventory, coin handling, change calculation.
☐	ATM	State plus Chain of Responsibility for note dispensing. Transaction integrity.
☐	Elevator System HIGH	Scheduling strategy, State, multiple elevators, request queuing. Concurrency-heavy.
☐	Library Management	CRUD-shaped modelling, membership rules, fines, search.
☐	LRU Cache HIGH	Bridges DSA and LLD — HashMap plus doubly linked list, then generalised to pluggable eviction policies.
☐	Cache with TTL and eviction policies	Strategy for LRU/LFU/FIFO, expiry handling, thread safety.
☐	Rate Limiter HIGH	Token bucket, sliding window; Strategy over algorithms; concurrency essential.
☐	Splitwise	Strategy for split types (equal, exact, percentage), balance settlement, graph-ish debt simplification.
☐	Digital Wallet	Transactions, atomicity, concurrency on balance updates.

☐	BookMyShow CRITICAL	The meatiest common problem. Seat locking under concurrency, payment Strategy, Observer for notifications, multi-entity modelling.
☐	Hotel Booking	Availability over date ranges, pricing rules, overbooking handling.
☐	Meeting Room Scheduler	Interval conflict detection, calendar modelling, notification.
☐	Task Scheduler	Command pattern, priority queues, recurring tasks, worker threads.
☐	Notification System	Observer-heavy; channel Strategy; retry and fallback logic.
☐	Logging Framework	Singleton, Chain of Responsibility over levels, Strategy over sinks.
☐	Inventory Management	Stock tracking, reservations, concurrency on decrements.
☐	Chess	Rich polymorphism across piece types, move validation, board state. Large — good for discussion rounds.
☐	Food Delivery (Swiggy/Zomato)	Multi-actor modelling, order lifecycle State, delivery assignment Strategy.
☐	Ride-sharing (Uber/Ola)	Advanced. Matching strategy, surge pricing, location handling, heavy concurrency.
☐	Traffic Signal / Amazon Locker	Smaller State-machine problems, useful as timed drills.

Phase 6 — Consolidation and Mock Rounds

Interleaved every ~5 sessions. This is the phase that converts knowledge into performance. Retrieval under pressure is a different skill from understanding.

✓	Drill	Why it's scheduled
☐	Cold re-design, no notes CRITICAL	Re-designing a problem from three weeks ago with nothing in front of you. Re-reading feels productive and isn't; retrieval is what makes recall reliable under stress.
☐	Rapid-fire pattern recognition	"Discount rules that change per campaign" → Strategy. Trains the mapping from requirement to pattern, which is what you actually do in a round.
☐	Timed machine coding, 90 minutes	In your own IDE, runnable, tested. The only way to find out whether you can actually finish.
☐	Full mock discussion round	You talk, design and code while being interrupted with follow-ups. Simulates the real cognitive load.
☐	Time-boxing practice	A common failure: 30 minutes on requirements, no time left to code. Practise the budget explicitly.
☐	Handling "I don't know"	Bluffing is transparent and costly. Saying what you'd look up, and reasoning from what you do know, scores far better.
☐	Recovering from "that's wrong"	Interviewers push back deliberately to see whether you defend reflexively or actually re-evaluate. Both extremes lose points.

Cross-Cutting Notes

Leverage your Spring Boot background

Spring is a dependency-injection container built on the exact patterns in Phase 3. `@Autowired` is constructor injection and Dependency Inversion. `@Service` / `@Repository` are layered architecture. Singleton is Spring's default bean scope. AOP is Proxy. `JpaRepository` is the Repository pattern. When each pattern comes up, tie it back to the framework you've already used — it converts abstract theory into something you've shipped.

Recurring habits to keep watching

- Computing a value and never printing or returning it — the single most frequent slip so far.
- `[]` indexing on a `List` instead of `.get(i)` — a C++ habit Java rejects.
- Java has no truthiness: `if (obj)` is invalid; write `if (obj == null)`, `if (list.isEmpty())`.
- Forgetting `public` on constructors and methods that form the class's contract.
- `.length` (arrays) vs `.size()` (collections) vs `.length()` (String).
- Passing a class name where an instance is needed — `new` is what makes an object.

Practice environment

Small syntax drills are fine anywhere. From Phase 3 onward, write code in IntelliJ or VS Code and actually run it — compiler errors teach Java faster than review comments, and machine coding rounds are judged on code that compiles and runs.

Progress tracker — tick each box as you complete it. Phases 0–2 are sequential; Phase 3 onward can interleave with Phase 5 problems, which is usually more effective than finishing all patterns first, since a pattern makes far more sense after you've felt the problem it solves.